Mini project report on

Personal Credit Health and Debt Repayment Planning System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS351A – DBMS Project

Submitted by:

Amoghaa R	**PES2UG24CS625**
Adarsha V	**PES2UG24CS621**

under the guidance of

Dr. Shilpa S

Assistant Professor

PES University

AUG - DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Personal Credit Health and Debt Repayment Planning System

is a bonafide work carried out by

Amoghaa R	**PES2UG24CS625**
Adarsha V	**PES2UG24CS621**

In partial fulfilment for the completion of fifth semester DBMS Project (UE24CS351A) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature

Dr. Shilpa S
Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Personal Credit Health and Debt Repayment Planning System** has been carried out by us under the guidance of **Dr. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester AUG – DEC 2026.

Amoghaa R	**PES2UG24CS625**	**<Signature>**
Adarsha V	**PES2UG24CS621**	**<Signature>**

ACKNOWLEDGEMENT

I would like to express my gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS351A - DBMS Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

The Personal Credit Health and Debt Repayment Planning System is a database-oriented project designed to centralize personal financial information and support effective debt repayment management.

The system maintains information related to users, financial profiles, loans, repayments, credit scores and repayment plans. A relational database is designed using an Entity Relationship model and implemented using MySQL.

The project demonstrates database concepts through DDL and DML statements, primary and foreign key constraints, SQL queries, transactions, stored procedures, triggers and indexing.

The proposed system provides a structured approach to managing financial records, monitoring repayment activities and organizing repayment plans. It demonstrates how database management concepts can be applied to a real-world personal financial management problem.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	9
2.	PROBLEM DEFINITION	10
3.	ER MODEL	11
4.	ER TO RELATIONAL MAPPING	12
5.	DDL STATEMENTS	13
6.	DML STATEMENTS	20
7.	QUERIES (SIMPLE QUERY AND UPDATE AND DELETE OPERATION, CORRELATED QUERY AND NESTED QUERY)	23
8.	STORED PROCEDURE, FUNCTIONS AND TRIGGERS	25
9.	FRONT END DEVELOPMENT	28
REFERENCES/BIBLIOGRAPHY		28

LIST OF TABLES

Table no	Title
5.1	USER Table
5.2	FINANCIAL_PROFILE Table
5.3	LOAN Table
5.4	REPAYMENT Table
5.5	REPAYMENT_PLAN Table
5.6	PLAN_PAYMENT Table

LIST OF FIGURES

Figure No	Title
6.1	Insertion of USER Record
6.2	Insertion of FINANCIAL_PROFILE Record
6.3	Insertion of LOAN Record
6.4	Insertion of REPAYMENT_PLAN Record
6.5	Insertion of PLAN_PAYMENT Record

1. INTRODUCTION

Personal financial management involves maintaining information about income, expenses, bank accounts, loans, credit scores and repayment activities. When this information is maintained separately, individuals may find it difficult to understand their overall financial position and manage repayment obligations effectively.

The Personal Credit Health and Debt Repayment Planning System is designed to provide a centralized database for organizing and managing personal financial information. It maintains user details, financial profiles, loan records, repayment history and repayment plans.

The system uses a relational database implemented in MySQL. Primary keys, foreign keys and integrity constraints maintain consistency between related records. SQL queries and database operations support financial analysis, repayment tracking and the management of repayment plans.

Objectives:

1. To maintain centralized personal financial records.
2. To store and manage loan and repayment information.
3. To monitor repayment status and outstanding amounts.
4. To maintain financial and credit-related information.
5. To support the creation and management of repayment plans.
6. To demonstrate database operations and advanced SQL concepts.

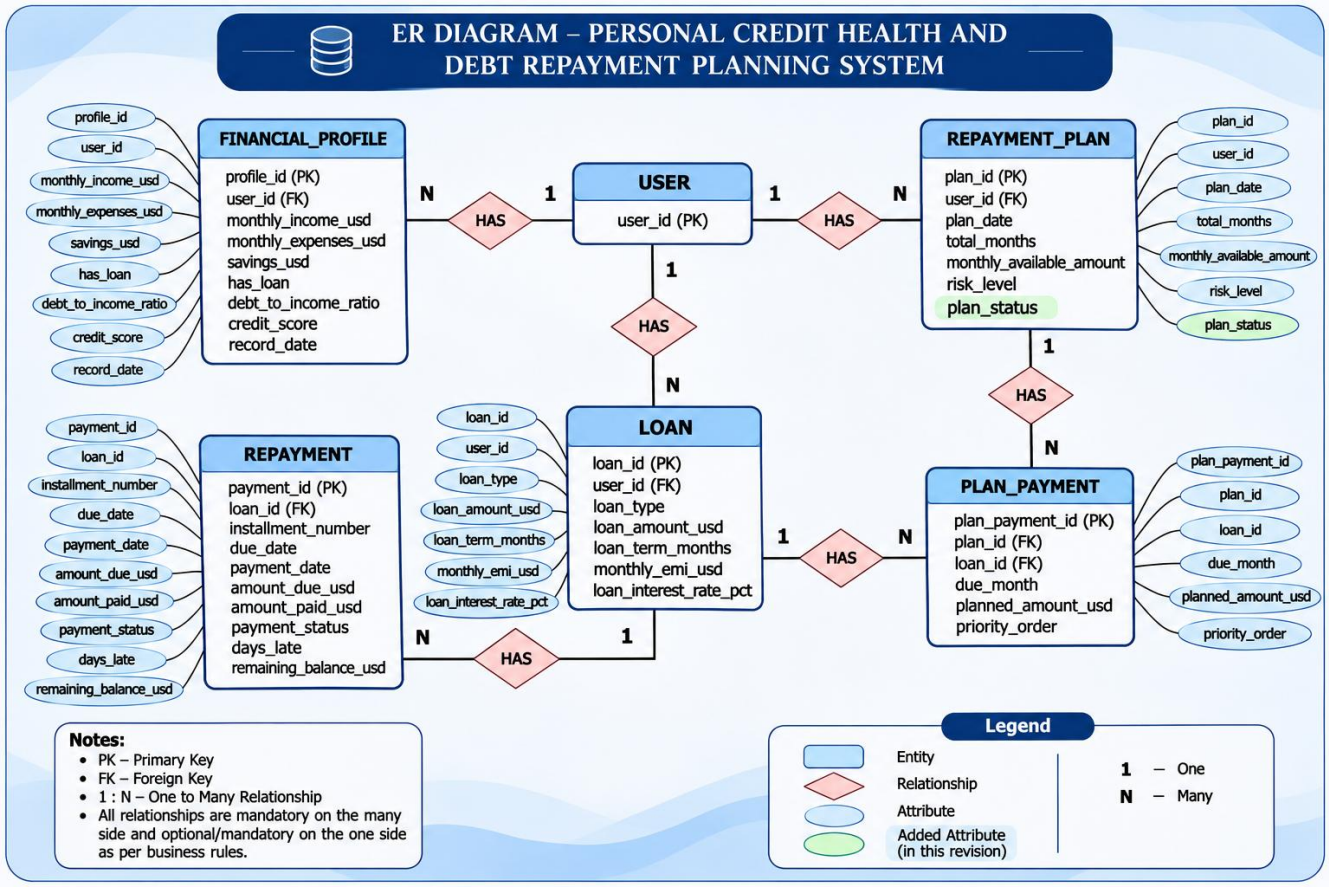
2. PROBLEM DEFINITION

Individuals often face difficulties managing their financial information because income, expenses, loans, credit scores and repayment records are maintained separately.

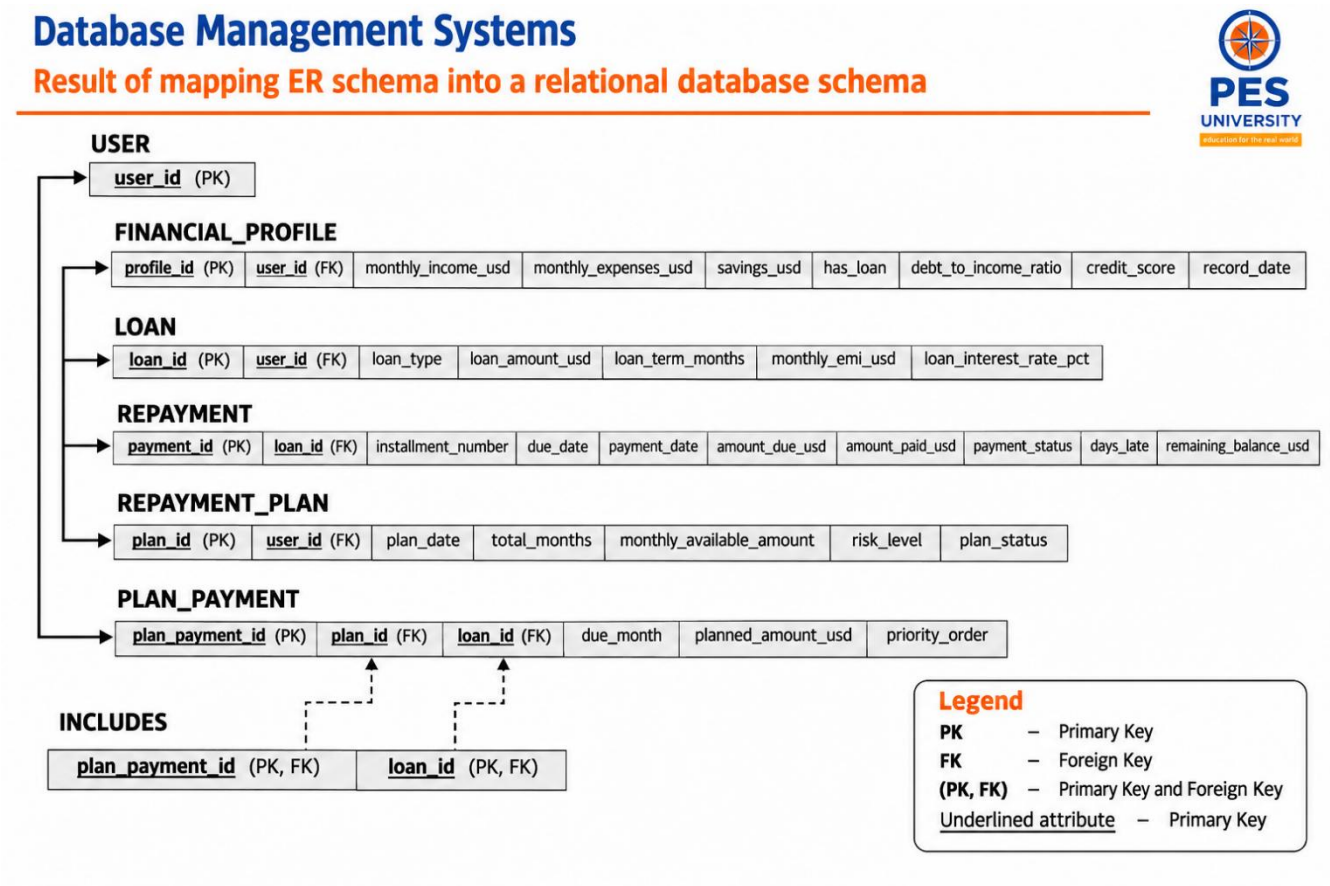
The absence of a centralized system makes it difficult to track outstanding loan amounts, monitor repayment deadlines, evaluate financial conditions and organize repayment priorities.
The proposed system addresses this problem by developing a relational database that stores personal financial information and maintains relationships between users, financial profiles, loans, repayments and repayment plans.

Proposed Solution:
The system provides a centralized MySQL database that allows financial records to be stored, updated, retrieved and analyzed using SQL operations. It also supports repayment planning and monitoring through related database tables.

3. ER MODEL


ER DIAGRAM – PERSONAL CREDIT HEALTH AND DEBT REPAYMENT PLANNING SYSTEM
FINANCIAL_PROFILE
profile_id (PK)
user_id (FK)
monthly_income_usd
monthly_expenses_usd
savings_usd
has_loan
debt_to_income_ratio
credit_score
record_date
profile_id
user_id
monthly_income_usd
monthly_expenses_usd
savings_usd
has_loan
debt_to_income_ratio
credit_score
record_date
N
HAS
1
USER
user_id (PK)
1
HAS
N
REPAYMENT_PLAN
plan_id (PK)
user_id (FK)
plan_date
total_months
monthly_available_amount
risk_level
plan_status
plan_id
user_id
plan_date
total_months
monthly_available_amount
risk_level
plan_status
1
HAS
N
LOAN
loan_id (PK)
user_id (FK)
loan_type
loan_amount_usd
loan_term_months
monthly_emi_usd
loan_interest_rate_pct
loan_id
user_id
loan_type
loan_amount_usd
loan_term_months
monthly_emi_usd
loan_interest_rate_pct
1
HAS
N
PLAN_PAYMENT
plan_payment_id (PK)
plan_id (FK)
loan_id (FK)
due_month
planned_amount_usd
priority_order
plan_payment_id
plan_id
loan_id
due_month
planned_amount_usd
priority_order
N
HAS
1
REPAYMENT
payment_id (PK)
loan_id (FK)
installment_number
due_date
payment_date
amount_due_usd
amount_paid_usd
payment_status
days_late
remaining_balance_usd
payment_id
loan_id
installment_number
due_date
payment_date
amount_due_usd
amount_paid_usd
payment_status
days_late
remaining_balance_usd
N
HAS
1
Notes:
PK – Primary Key
FK – Foreign Key
1 : N – One to Many Relationship
All relationships are mandatory on the many side and optional/mandatory on the one side as per business rules.
Legend
Entity
Relationship
Attribute
Added Attribute (in this revision)
1 – One
N – Many

4. ER TO RELATIONAL MAPPING



5. DDL STATEMENTS

STATEMENTS WITH SCREEN SHOTS OF THE TABLE CREATION

5.1 Database Creation and Selection

```
mysql> CREATE DATABASE IF NOT EXISTS credit_health_db;
Query OK, 1 row affected, 1 warning (0.04 sec)

mysql> USE credit_health_db;
Database changed
mysql>
```

5.2 Creation of USER Table

```
mysql> CREATE TABLE IF NOT EXISTS `user` (
    ->     user_id VARCHAR(10) NOT NULL,
    ->     PRIMARY KEY (user_id)
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> DESCRIBE `user`;
+---------+-------------+------+-----+---------+-------+
| Field   | Type        | Null | Key | Default | Extra |
+---------+-------------+------+-----+---------+-------+
| user_id | varchar(10) | NO   | PRI | NULL    |       |
+---------+-------------+------+-----+---------+-------+
1 row in set (0.00 sec)
```

5.3 Creation of FINANCIAL_PROFILE Table

```
mysql> CREATE TABLE IF NOT EXISTS financial_profile (
    ->     profile_id INT NOT NULL AUTO_INCREMENT,
    ->     user_id VARCHAR(10) NOT NULL,
    ->     monthly_income_usd DECIMAL(12,2) NOT NULL,
    ->     monthly_expenses_usd DECIMAL(12,2) NOT NULL,
    ->     savings_usd DECIMAL(12,2) NOT NULL,
    ->     has_loan VARCHAR(3) NOT NULL,
    ->     debt_to_income_ratio DECIMAL(6,2) NOT NULL,
    ->     credit_score INT NOT NULL,
    ->     record_date DATE NOT NULL,
    ->     PRIMARY KEY (profile_id),
    ->     CONSTRAINT fk_financial_profile_user
    ->         FOREIGN KEY (user_id) REFERENCES `user` (user_id),
    ->     CONSTRAINT chk_profile_income CHECK (monthly_income_usd >= 0),
    ->     CONSTRAINT chk_profile_expenses CHECK (monthly_expenses_usd >= 0),
    ->     CONSTRAINT chk_profile_savings CHECK (savings_usd >= 0),
    ->     CONSTRAINT chk_profile_has_loan CHECK (has_loan IN ('Yes','No')),
    ->     CONSTRAINT chk_profile_credit_score CHECK (credit_score BETWEEN 300 AND 850)
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> DESCRIBE financial_profile;
+---------------------+--------------+------+-----+---------+----------------+
| Field               | Type         | Null | Key | Default | Extra          |
+---------------------+--------------+------+-----+---------+----------------+
| profile_id          | int          | NO   | PRI | NULL    | auto_increment |
| user_id             | varchar(10)  | NO   | MUL | NULL    |                |
| monthly_income_usd  | decimal(12,2)| NO   |     | NULL    |                |
| monthly_expenses_usd| decimal(12,2)| NO   |     | NULL    |                |
| savings_usd         | decimal(14,2)| NO   |     | NULL    |                |
| has_loan            | varchar(3)   | NO   |     | NULL    |                |
| debt_to_income_ratio| decimal(6,2) | NO   |     | NULL    |                |
| credit_score        | int          | NO   |     | NULL    |                |
| record_date         | date         | NO   |     | NULL    |                |
+---------------------+--------------+------+-----+---------+----------------+
9 rows in set (0.00 sec)
```

5.4 Creation of LOAN Table

```
mysql> CREATE TABLE IF NOT EXISTS loan (
    ->     loan_id VARCHAR(10) NOT NULL,
    ->     user_id VARCHAR(10) NOT NULL,
    ->     loan_type VARCHAR(20) NOT NULL,
    ->     loan_amount_usd DECIMAL(14,2) NOT NULL,
    ->     loan_term_months INT NOT NULL,
    ->     monthly_emi_usd DECIMAL(12,2) NOT NULL,
    ->     loan_interest_rate DECIMAL(6,2) NOT NULL,
    ->     PRIMARY KEY (loan_id),
    ->     CONSTRAINT fk_loan_user
    ->         FOREIGN KEY (user_id) REFERENCES `user` (user_id),
    ->     CONSTRAINT chk_loan_amount CHECK (loan_amount_usd > 0),
    ->     CONSTRAINT chk_loan_term CHECK (loan_term_months > 0),
    ->     CONSTRAINT chk_loan_emi CHECK (monthly_emi_usd >= 0),
    ->     CONSTRAINT chk_loan_interest CHECK (loan_interest_rate >= 0)
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> DESCRIBE loan;
+----------------------+---------------+------+-----+---------+-------+
| Field                | Type          | Null | Key | Default | Extra |
+----------------------+---------------+------+-----+---------+-------+
| loan_id              | varchar(10)   | NO   | PRI | NULL    |       |
| user_id              | varchar(10)   | NO   | MUL | NULL    |       |
| loan_type            | varchar(20)   | NO   |     | NULL    |       |
| loan_amount_usd      | decimal(14,2) | NO   |     | NULL    |       |
| loan_term_months     | int           | NO   |     | NULL    |       |
| monthly_emi_usd      | decimal(12,2) | NO   |     | NULL    |       |
| loan_interest_rate_pct | decimal(6,2)  | NO   |     | NULL    |       |
+----------------------+---------------+------+-----+---------+-------+
7 rows in set (0.00 sec)
```

5.5 Creation of REPAYMENT Table

```
mysql> CREATE TABLE IF NOT EXISTS repayment (
    ->     payment_id VARCHAR(20) NOT NULL,
    ->     loan_id VARCHAR(10) NOT NULL,
    ->     installment_number INT NOT NULL,
    ->     due_date DATE NOT NULL,
    ->     payment_date DATE NULL,
    ->     amount_due_usd DECIMAL(12,2) NOT NULL,
    ->     amount_paid_usd DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    ->     payment_status VARCHAR(20) NOT NULL,
    ->     days_late INT NOT NULL,
    ->     remaining_balance_usd DECIMAL(14,2) NOT NULL,
    ->     PRIMARY KEY (payment_id),
    ->     CONSTRAINT fk_repayment_loan
    ->         FOREIGN KEY (loan_id) REFERENCES loan (loan_id),
    ->     CONSTRAINT chk_repayment_installment CHECK (installment_number > 0),
    ->     CONSTRAINT chk_repayment_due CHECK (amount_due_usd >= 0),
    ->     CONSTRAINT chk_repayment_paid CHECK (amount_paid_usd >= 0),
    ->     CONSTRAINT chk_repayment_late CHECK (days_late >= 0),
    ->     CONSTRAINT chk_repayment_balance CHECK (remaining_balance_usd >= 0)
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> DESCRIBE repayment;
+----------------------+---------------+------+-----+---------+-------+
| Field                | Type          | Null | Key | Default | Extra |
+----------------------+---------------+------+-----+---------+-------+
| payment_id           | varchar(20)   | NO   | PRI | NULL    |       |
| loan_id              | varchar(10)   | NO   | MUL | NULL    |       |
| installment_number   | int           | NO   |     | NULL    |       |
| due_date             | date          | NO   | MUL | NULL    |       |
| payment_date         | date          | YES  |     | NULL    |       |
| amount_due_usd       | decimal(12,2) | NO   |     | NULL    |       |
| amount_paid_usd      | decimal(12,2) | NO   |     | 0.00    |       |
| payment_status       | varchar(20)   | NO   | MUL | NULL    |       |
| days_late            | int           | NO   |     | 0       |       |
| remaining_balance_usd | decimal(14,2) | NO   |     | NULL    |       |
+----------------------+---------------+------+-----+---------+-------+
10 rows in set (0.00 sec)
```

5.6 Creation of REPAYMENT_PLAN Table

```
mysql> CREATE TABLE IF NOT EXISTS repayment_plan (
    ->     plan_id INT NOT NULL AUTO_INCREMENT,
    ->     user_id VARCHAR(10) NOT NULL,
    ->     plan_date DATE NOT NULL,
    ->     total_months INT NOT NULL,
    ->     monthly_available_amount DECIMAL(12,2) NOT NULL,
    ->     risk_level VARCHAR(10) NOT NULL,
    ->     PRIMARY KEY (plan_id),
    ->     CONSTRAINT fk_repayment_plan_user
    ->         FOREIGN KEY (user_id) REFERENCES `user` (user_id),
    ->     CONSTRAINT chk_plan_months CHECK (total_months > 0),
    ->     CONSTRAINT chk_plan_available CHECK (monthly_available_amount >= 0)
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> DESCRIBE repayment_plan;
+--------------------------+---------------+------+-----+---------+----------------+
| Field                    | Type          | Null | Key | Default | Extra          |
+--------------------------+---------------+------+-----+---------+----------------+
| plan_id                  | int           | NO   | PRI | NULL    | auto_increment |
| user_id                  | varchar(10)   | NO   | MUL | NULL    |                |
| plan_date                | date          | NO   |     | NULL    |                |
| total_months             | int           | NO   |     | NULL    |                |
| monthly_available_amount | decimal(12,2) | NO   |     | NULL    |                |
| risk_level               | varchar(10)   | NO   |     | NULL    |                |
| plan_status              | varchar(15)   | NO   |     | Active  |                |
+--------------------------+---------------+------+-----+---------+----------------+
7 rows in set (0.00 sec)
```

5.7 Creation of PLAN_PAYMENT Table

```
mysql> CREATE TABLE IF NOT EXISTS plan_payment (
    ->     plan_payment_id INT NOT NULL AUTO_INCREMENT,
    ->     plan_id INT NOT NULL,
    ->     loan_id VARCHAR(10) NOT NULL,
    ->     due_month DATE NOT NULL,
    ->     planned_amount_usd DECIMAL(12,2) NOT NULL,
    ->     priority_order INT NOT NULL,
    ->     PRIMARY KEY (plan_payment_id),
    ->     CONSTRAINT fk_plan_payment_plan
    ->         FOREIGN KEY (plan_id) REFERENCES repayment_plan (plan_id),
    ->     CONSTRAINT fk_plan_payment_loan
    ->         FOREIGN KEY (loan_id) REFERENCES loan (loan_id),
    ->     CONSTRAINT chk_planned_amount CHECK (planned_amount_usd >= 0),
    ->     CONSTRAINT chk_priority_order CHECK (priority_order > 0)
    -> ) ENGINE=InnoDB;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> DESCRIBE plan_payment;
+--------------------+---------------+------+-----+---------+----------------+
| Field              | Type          | Null | Key | Default | Extra          |
+--------------------+---------------+------+-----+---------+----------------+
| plan_payment_id    | int           | NO   | PRI | NULL    | auto_increment |
| plan_id            | int           | NO   | MUL | NULL    |                |
| loan_id            | varchar(10)   | NO   | MUL | NULL    |                |
| due_month          | date          | NO   |     | NULL    |                |
| planned_amount_usd | decimal(12,2) | NO   |     | NULL    |                |
| priority_order     | int           | NO   |     | NULL    |                |
+--------------------+---------------+------+-----+---------+----------------+
6 rows in set (0.00 sec)
```

6. DML STATEMENTS

STATEMENTS WITH SCREEN SHOTS OF THE TABLE WITH INSERTED VALUES

6.1 Insertion of USER Record

```
mysql> INSERT INTO `user` (user_id)
    -> VALUES ('U03941')
    -> ON DUPLICATE KEY UPDATE user_id = VALUES(user_id);
Query OK, 0 rows affected, 1 warning (0.02 sec)

mysql> SELECT * FROM `user`
    -> WHERE user_id = 'U03941';
+---------+
| user_id |
+---------+
| U03941  |
+---------+
1 row in set (0.00 sec)
```

6.2 Insertion of FINANCIAL_PROFILE Record

```
mysql> INSERT INTO financial_profile
    ->     (user_id, monthly_income_usd, monthly_expenses_usd, savings_usd,
    ->      has_loan, debt_to_income_ratio, credit_score, record_date)
    -> SELECT 'U03941', 5000.00, 2500.00, 4000.00,
    ->        'Yes', 35.00, 720, '2026-08-24'
    -> WHERE NOT EXISTS (
    ->     SELECT 1 FROM financial_profile
    ->     WHERE user_id = 'U03941'
    ->     AND record_date = '2026-08-24'
    -> );
Query OK, 1 row affected (0.04 sec)
Records: 1  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM financial_profile
    -> WHERE user_id = 'U03941';
+------------+---------+--------------------+---------------------+-------------+----------+---------------------+--------------+-------------+
| profile_id | user_id | monthly_income_usd | monthly_expenses_usd | savings_usd | has_loan | debt_to_income_ratio | credit_score | record_date |
+------------+---------+--------------------+---------------------+-------------+----------+---------------------+--------------+-------------+
|       1573 | U03941  |            3936.35 |             1598.89 |   415121.73 | Yes      |                0.20 |          300 | 2022-11-28  |
|      16384 | U03941  |            5000.00 |             2500.00 |     4000.00 | Yes      |               35.00 |          720 | 2026-08-24  |
+------------+---------+--------------------+---------------------+-------------+----------+---------------------+--------------+-------------+
2 rows in set (0.00 sec)
```

6.3 Insertion of LOAN Record

```
mysql> INSERT INTO loan
    ->     (loan_id, user_id, loan_type, loan_amount_usd, loan_term_months,
    ->      monthly_emi_usd, loan_interest_rate_pct)
    -> VALUES
    ->     ('L03941', 'U03941', 'Personal', 10000.00, 24, 500.00, 8.50)
    -> ON DUPLICATE KEY UPDATE loan_id = VALUES(loan_id);
Query OK, 0 rows affected, 1 warning (0.02 sec)

mysql> SELECT * FROM loan
    -> WHERE loan_id = 'L03941';
+---------+---------+-----------+-----------------+------------------+-----------------+------------------------+
| loan_id | user_id | loan_type | loan_amount_usd | loan_term_months | monthly_emi_usd | loan_interest_rate_pct |
+---------+---------+-----------+-----------------+------------------+-----------------+------------------------+
| L03941  | U03941  | Education |       114456.85 |              360 |          794.82 |                   7.43 |
+---------+---------+-----------+-----------------+------------------+-----------------+------------------------+
1 row in set (0.00 sec)
```

6.4 Insertion of REPAYMENT_PLAN Record

```
mysql> INSERT INTO repayment_plan
    ->     (plan_id, user_id, plan_date, total_months,
    ->      monthly_available_amount, risk_level, plan_status)
    -> VALUES
    ->     (1, 'U03941', '2026-08-24', 12, 2339.46, 'High', 'Active')
    -> ON DUPLICATE KEY UPDATE
    ->     user_id = VALUES(user_id),
    ->     plan_date = VALUES(plan_date),
    ->     total_months = VALUES(total_months),
    ->     monthly_available_amount = VALUES(monthly_available_amount),
    ->     risk_level = VALUES(risk_level),
    ->     plan_status = VALUES(plan_status);
Query OK, 2 rows affected, 6 warnings (0.02 sec)

mysql> SELECT plan_id, user_id, plan_date, total_months,
    ->        monthly_available_amount, risk_level, plan_status
    -> FROM repayment_plan
    -> WHERE plan_id = 1;
+---------+---------+------------+--------------+--------------------------+------------+-------------+
| plan_id | user_id | plan_date  | total_months | monthly_available_amount | risk_level | plan_status |
+---------+---------+------------+--------------+--------------------------+------------+-------------+
|       1 | U03941  | 2026-08-24 |           12 |                  2339.46 | High       | Active      |
+---------+---------+------------+--------------+--------------------------+------------+-------------+
1 row in set (0.00 sec)
```

6.5 Insertion of PLAN_PAYMENT Record

```
mysql> INSERT INTO plan_payment
    ->     (plan_id, loan_id, due_month, planned_amount_usd, priority_order)
    -> VALUES
    ->     (1, 'L03941', '2026-09-01', 794.82, 1);
Query OK, 1 row affected (0.03 sec)

mysql> SELECT plan_payment_id, plan_id, loan_id, due_month,
    ->        planned_amount_usd, priority_order
    -> FROM plan_payment
    -> WHERE plan_id = 1;
+-----------------+---------+---------+------------+--------------------+----------------+
| plan_payment_id | plan_id | loan_id | due_month  | planned_amount_usd | priority_order |
+-----------------+---------+---------+------------+--------------------+----------------+
|               2 |       1 | L03941  | 2026-09-01 |             874.30 |              1 |
|               3 |       1 | L03941  | 2026-09-01 |             874.30 |              1 |
|               4 |       1 | L03941  | 2026-09-01 |             794.82 |              1 |
+-----------------+---------+---------+------------+--------------------+----------------+
3 rows in set (0.00 sec)
```

7. QUERIES

7.1 Grouping Loans by Type Using Aggregate Functions

```
mysql> SELECT
    ->     loan_type,
    ->     COUNT(*) AS total_loans,
    ->     SUM(loan_amount_usd) AS total_loan_amount,
    ->     AVG(loan_amount_usd) AS average_loan_amount
    -> FROM loan
    -> GROUP BY loan_type;
+-----------+-------------+-------------------+---------------------+
| loan_type | total_loans | total_loan_amount | average_loan_amount |
+-----------+-------------+-------------------+---------------------+
| Education |        3275 |      826544640.39 |       252380.042867 |
| Business  |        3261 |      808450183.75 |       247914.806424 |
| Car       |        3175 |      801443117.30 |       252423.029071 |
| Home      |        3284 |      809682258.38 |       246553.671857 |
+-----------+-------------+-------------------+---------------------+
4 rows in set (0.04 sec)
```

7.2 Updating the Planned Payment Amount

```
mysql> UPDATE plan_payment
    -> SET planned_amount_usd = 874.30
    -> WHERE plan_id = 1
    ->   AND loan_id = 'L03941'
    ->   AND due_month = '2026-09-01';
Query OK, 1 row affected (0.02 sec)
Rows matched: 3  Changed: 1  Warnings: 0

mysql> SELECT *
    -> FROM plan_payment
    -> WHERE plan_id = 1;
+-----------------+---------+---------+------------+--------------------+----------------+
| plan_payment_id | plan_id | loan_id | due_month  | planned_amount_usd | priority_order |
+-----------------+---------+---------+------------+--------------------+----------------+
|               2 |       1 | L03941  | 2026-09-01 |             874.30 |              1 |
|               3 |       1 | L03941  | 2026-09-01 |             874.30 |              1 |
|               4 |       1 | L03941  | 2026-09-01 |             874.30 |              1 |
+-----------------+---------+---------+------------+--------------------+----------------+
3 rows in set (0.00 sec)
```

7.3 Deleting a Planned Payment Record

```
mysql> DELETE FROM plan_payment
    -> WHERE plan_id = 1
    ->   AND loan_id = 'L03941';
Query OK, 3 rows affected (0.01 sec)

mysql> SELECT *
    -> FROM plan_payment
    -> WHERE plan_id = 1;
Empty set (0.00 sec)
```

7.4 Retrieving Loan Details Using a Correlated Subquery

```
mysql> SELECT
    ->     l.loan_id,
    ->     l.user_id,
    ->     l.loan_type,
    ->     l.loan_amount_usd,
    ->     (
    ->         SELECT COUNT(*)
    ->         FROM repayment r
    ->         WHERE r.loan_id = l.loan_id
    ->     ) AS total_repayments
    -> FROM loan l
    -> LIMIT 10;
+---------+---------+-----------+-----------------+------------------+
| loan_id | user_id | loan_type | loan_amount_usd | total_repayments |
+---------+---------+-----------+-----------------+------------------+
| L00002  | U00002  | Education |       146323.34 |               13 |
| L00004  | U00004  | Business  |        93242.37 |               13 |
| L00006  | U00006  | Car       |       407915.25 |               13 |
| L00007  | U00007  | Car       |       296057.60 |               13 |
| L00010  | U00010  | Education |       324197.37 |               13 |
| L00011  | U00011  | Education |       127703.79 |               13 |
| L00014  | U00014  | Car       |       404941.02 |               13 |
| L00021  | U00021  | Business  |       425717.20 |               13 |
| L00022  | U00022  | Home      |       180386.08 |               13 |
| L00023  | U00023  | Education |        78365.77 |               13 |
+---------+---------+-----------+-----------------+------------------+
10 rows in set (0.00 sec)
```

7.5 Retrieving Financial Profiles Using a Nested Query

```
mysql> SELECT
    ->     user_id,
    ->     credit_score,
    ->     monthly_income_usd,
    ->     monthly_expenses_usd,
    ->     savings_usd
    -> FROM financial_profile
    -> WHERE user_id IN
    -> (
    ->     SELECT user_id
    ->     FROM loan
    -> )
    -> LIMIT 10;
+---------+--------------+--------------------+----------------------+-------------+
| user_id | credit_score | monthly_income_usd | monthly_expenses_usd | savings_usd |
+---------+--------------+--------------------+----------------------+-------------+
| U00002  |          543 |            3531.73 |              2367.99 |   260869.10 |
| U00004  |          461 |            5894.88 |              4440.12 |   304815.51 |
| U00006  |          718 |            4051.00 |              2244.42 |   194901.41 |
| U00007  |          761 |             983.69 |               332.62 |    37522.56 |
| U00010  |          558 |            5216.18 |              4369.36 |   203320.50 |
| U00011  |          412 |            3677.43 |              2247.92 |   311546.91 |
| U00014  |          788 |            5546.28 |              3630.24 |   453101.38 |
| U00021  |          622 |            8237.77 |              7316.78 |    83622.72 |
| U00022  |          707 |            7093.07 |              3500.65 |   723648.99 |
| U00023  |          641 |            1694.87 |               604.03 |   118470.87 |
+---------+--------------+--------------------+----------------------+-------------+
10 rows in set (0.01 sec)
```

8. STORED PROCEDURES, FUNCTIONS AND TRIGGERS

8.1 STORED PROCEDURES OR FUNCTIONS

8.1.1 Stored Procedure Creation

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetLoanRepaymentDetails(
    ->     IN p_loan_id VARCHAR(10)
    -> )
    -> BEGIN
    ->     SELECT
    ->         loan_id,
    ->         installment_number,
    ->         due_date,
    ->         payment_date,
    ->         amount_due_usd,
    ->         amount_paid_usd,
    ->         payment_status,
    ->         days_late,
    ->         remaining_balance_usd
    ->     FROM repayment
    ->     WHERE loan_id = p_loan_id;
    -> END //
Query OK, 0 rows affected (0.07 sec)
```

8.1.2 Stored Procedure Execution

```
mysql> CALL GetLoanRepaymentDetails('L00002');
+---------+-------------------+------------+--------------+----------------+-----------------+----------------+-----------+-----------------------+
| loan_id | installment_number | due_date   | payment_date | amount_due_usd | amount_paid_usd | payment_status | days_late | remaining_balance_usd |
+---------+-------------------+------------+--------------+----------------+-----------------+----------------+-----------+-----------------------+
| L00002  |                 1 | 2021-02-13 | 2021-03-09   |        4953.50 |         4953.50 | Paid Late      |        24 |             141369.84 |
| L00002  |                 2 | 2021-03-13 | 2021-03-13   |        4953.50 |         4953.50 | Paid           |         0 |             136416.34 |
| L00002  |                 3 | 2021-04-13 | 2021-05-03   |        4953.50 |         4953.50 | Paid Late      |        20 |             131462.84 |
| L00002  |                 4 | 2021-05-13 | 2021-05-13   |        4953.50 |         4953.50 | Paid           |         0 |             126509.34 |
| L00002  |                 5 | 2021-06-13 | 2021-06-13   |        4953.50 |         4953.50 | Paid           |         0 |             121555.84 |
| L00002  |                 6 | 2021-07-13 | NULL         |        4953.50 |            0.00 | Missed         |        25 |             121555.84 |
| L00002  |                 7 | 2021-08-13 | 2021-09-07   |        4953.50 |         4953.50 | Paid Late      |        25 |             116602.34 |
| L00002  |                 8 | 2021-09-13 | 2021-09-30   |        4953.50 |         4953.50 | Paid Late      |        17 |             111648.84 |
| L00002  |                 9 | 2021-10-13 | 2021-10-13   |        4953.50 |         4953.50 | Paid           |         0 |             106695.34 |
| L00002  |                10 | 2021-11-13 | 2021-11-13   |        4953.50 |         4953.50 | Paid           |         0 |             101741.84 |
| L00002  |                11 | 2021-12-13 | 2021-12-13   |        4953.50 |         4953.50 | Paid           |         0 |              96788.34 |
| L00002  |                12 | 2022-01-13 | NULL         |        4953.50 |            0.00 | Missed         |        27 |              96788.34 |
| L00002  |                13 | 2022-03-13 | NULL         |        4953.50 |            0.00 | Upcoming       |         0 |              96788.34 |
+---------+-------------------+------------+--------------+----------------+-----------------+----------------+-----------+-----------------------+
13 rows in set (0.01 sec)

Query OK, 0 rows affected (0.06 sec)
```

8.1.3 Stored Procedure Verification

```
mysql> DELIMITER ;
mysql> SHOW PROCEDURE STATUS
    -> WHERE Db = 'credit_health_db';
+------------------+--------------------------+-----------+----------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+-----------------------+
| Db               | Name                     | Type      | Definer        | Modified            | Created             | Security_type | Comment | character_set_client | collation_connection | Database Collation    |
+------------------+--------------------------+-----------+----------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+-----------------------+
| credit_health_db | GetLoanRepaymentDetails  | PROCEDURE | root@localhost | 2026-11-01 21:32:07 | 2026-11-01 21:32:07 | DEFINER       |         | cp850                | cp850_general_ci     | utf8mb4_0900_ai_ci    |
| credit_health_db | GetUserCreditHealth      | PROCEDURE | root@localhost | 2026-08-24 20:30:47 | 2026-08-24 20:30:47 | DEFINER       |         | cp850                | cp850_general_ci     | utf8mb4_0900_ai_ci    |
+------------------+--------------------------+-----------+----------------+---------------------+---------------------+---------------+---------+----------------------+----------------------+-----------------------+
2 rows in set (0.01 sec)
```

8.2 TRIGGERS

8.2.1 Trigger to Automatically Calculate Days Late

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER Calculate_Days_Late
    -> BEFORE INSERT ON repayment
    -> FOR EACH ROW
    -> BEGIN
    ->     SET NEW.days_late =
    ->         CASE
    ->             WHEN NEW.payment_date IS NOT NULL
    ->             THEN GREATEST(
    ->                 DATEDIFF(NEW.payment_date, NEW.due_date), 0
    ->             )
    ->             ELSE GREATEST(
    ->                 DATEDIFF(CURDATE(), NEW.due_date), 0
    ->             )
    ->         END;
    -> END //
Query OK, 0 rows affected (0.04 sec)
```

8.2.2 Trigger Verification

```
mysql> SHOW TRIGGERS
    -> WHERE `Table` = 'repayment'\G
*************************** 1. row ***************************
             Trigger: Calculate_Days_Late
               Event: INSERT
               Table: repayment
           Statement: BEGIN
    SET NEW.days_late =
        CASE
            WHEN NEW.payment_date IS NOT NULL
            THEN GREATEST(
                DATEDIFF(NEW.payment_date, NEW.due_date), 0
            )
            ELSE GREATEST(
                DATEDIFF(CURDATE(), NEW.due_date), 0
            )
        END;
END
              Timing: BEFORE
             Created: 2026-11-01 21:41:55.55
            sql_mode: ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE
,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION
             Definer: root@localhost
character_set_client: cp850
collation_connection: cp850_general_ci
  Database Collation: utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

9. FRONT END DEVELOPEMNT

REFERENCES

[1]

GitHub Link: